STBL Stellar Contracts Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

SBSecurity (Blckhv & Slavcheww)

BengalCatBalu

January 30, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **3**

7 Findings **5**

7.1 Critical Risk . . . 5
7.1.1 Missing caller auth allows access control takeover . . . 5
7.1.2 Yield distribution blocked due to wrong `assert_issuer` usage . . . 6
7.2 High Risk . . . 10
7.2.1 Incorrect storage Key in `increase_nft_counter` causes NFT ownership overwrites . . . 10
7.2.2 Missing Decimal Normalization Results in Incorrect USST Minting Amounts . . . 11
7.2.3 YLD Token Burn Uses `asset_id` Instead of `token_id` for Ownership Verification . . . 12
7.2.4 Inverted YLD Validity Check Blocks All Reward Claims . . . 13
7.2.5 Missing Token Approval Breaks Yield Distribution . . . 15
7.3 Medium Risk . . . 17
7.3.1 PT Vault distributeYield Missing Treasury Authorization . . . 17
7.3.2 Persistent Storage TTL Expiry Causes Security Bypass and State Corruption . . . 18
7.3.3 Withdraw Fee Incorrectly Included in Staking Balance . . . 21
7.4 Low Risk . . . 23
7.4.1 Oracle Initialized with Zero Price Decimals Breaks Price Calculations . . . 23
7.4.2 No Slippage Protection on Deposit and Withdraw Operations . . . 23
7.4.3 No Validation That Asset Token Supports Required Interface . . . 25
7.4.4 Emergency Withdraw Does Not Reset Vault Accounting State . . . 25
7.4.5 `asset-oracle` is missing price validation . . . 27
7.4.6 Single Oracle Dependency Can Freeze User Funds . . . 27
7.5 Informational . . . 29
7.5.1 Unused local `UPGRADER_ROLE` in per asset contracts . . . 29
7.5.2 Unused Pause Functionality in Asset Contracts . . . 29
7.5.3 Vault Solvency Check Blocks User Withdrawals After Yield Distribution . . . 30
7.5.4 Withdraw Fee Locked at Deposit Time . . . 30
7.6 Gas Optimization . . . 32
7.6.1 Redundant `is_active` Checks in Deposit and Withdraw Flows . . . 32

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

STBL Protocol is a yield-bearing stablecoin platform that tokenizes Real World Assets (RWAs) through a dual-token architecture whose EVM implementation was previously audited by Cyfrin.

The focus of this audit was STBL's new Stellar / Soroban implementation.

5 Audit Scope

The audit scope was limited to:

```
contracts/asset-issuer/src/lib.rs
contracts/asset-oracle/src/lib.rs
contracts/asset-vault/src/lib.rs
contracts/asset-yield-distributor/src/lib.rs
contracts/core/src/lib.rs
contracts/register/src/lib.rs
contracts/stbl-token/src/lib.rs
contracts/usst-token/src/lib.rs
contracts/yld-token/src/lib.rs
libraries/access-control/src/lib.rs
libraries/access-control/src/macros.rs
libraries/pauser/src/lib.rs
libraries/pauser/src/macros.rs
libraries/stbl-core-types/src/asset_definition.rs
libraries/stbl-core-types/src/decimal_converter.rs
libraries/stbl-core-types/src/lib.rs
```

6 Executive Summary

Over the course of 8 days, the Cyfrin team conducted an audit on the STBL Stellar Contracts smart contracts provided by STBL. In this period, a total of 21 issues were found.

The findings consist of 2 Critical, 5 High, 3 Medium and 6 Low severity issues with the remainder being informational.

Post Audit Recommendations

Due to the significant number of Critical & High severity findings it is statistically likely that more serious vulnerabilities still remain. Hence it is recommended that prior to deploying significant capital on-chain in a production environment, another audit be conducted during which no Critical or High severity findings should be found.

Summary

Project Name	STBL Stellar Contracts
Repository	stellar-core-contracts
Commit	6090b4b47446...
Fix Commit	39a9e3b4e9eb...
Audit Timeline	Jan 21st - Jan 30th, 2026
Methods	Manual Review

Issues Found

Critical Risk	2
High Risk	5
Medium Risk	3
Low Risk	6
Informational	4
Gas Optimizations	1
Total Issues	21

Summary of Findings

[C-1] Missing caller auth allows access control takeover	Resolved
[C-2] Yield distribution blocked due to wrong `assert_issuer` usage	Resolved
[H-1] Incorrect storage Key in `increase_nft_counter` causes NFT ownership overwrites	Resolved
[H-2] Missing Decimal Normalization Results in Incorrect USST Minting Amounts	Resolved

[H-3] YLD Token Burn Uses `asset_id` Instead of `token_id` for Ownership Verification	Resolved
[H-4] Inverted YLD Validity Check Blocks All Reward Claims	Resolved
[H-5] Missing Token Approval Breaks Yield Distribution	Resolved
[M-1] PT Vault distributeYield Missing Treasury Authorization	Resolved
[M-2] Persistent Storage TTL Expiry Causes Security Bypass and State Corruption	Resolved
[M-3] Withdraw Fee Incorrectly Included in Staking Balance	Resolved
[L-1] Oracle Initialized with Zero Price Decimals Breaks Price Calculations	Resolved
[L-2] No Slippage Protection on Deposit and Withdraw Operations	Resolved
[L-3] No Validation That Asset Token Supports Required Interface	Acknowledged
[L-4] Emergency Withdraw Does Not Reset Vault Accounting State	Resolved
[L-5] `asset-oracle` is missing price validation	Resolved
[L-6] Single Oracle Dependency Can Freeze User Funds	Acknowledged
[I-1] Unused local `UPGRADER_ROLE` in per asset contracts	Resolved
[I-2] Unused Pause Functionality in Asset Contracts	Resolved
[I-3] Vault Solvency Check Blocks User Withdrawals After Yield Distribution	Acknowledged
[I-4] Withdraw Fee Locked at Deposit Time	Resolved
[G-1] Redundant `is_active` Checks in Deposit and Withdraw Flows	Resolved

7 Findings

7.1 Critical Risk

7.1.1 Missing caller auth allows access control takeover

Description: Due to a missing caller authentication, anyone can take over the roles and seize the contracts. The following functions are missing `require_auth`:

- `grant_role`
- `revoke_role`
- `renounce_role` (`assert_eq!(caller, account);` doesn't protect from this vector)
- `update_role_admin`
- `pause`
- `unpause`

Impact: All these contracts are vulnerable, since they expose macro functions from the `access-control` and `pauser` libraries and vulnerable functions that lack the check can be invoked externally, by anyone: `UsstToken`, `StblCore`, `AssetOracle`, `StblToken`, `AssetVault`, `YldToken`, `StblRegister`, `AssetIssuer`, `AssetYieldDistributor`.

All the roles can be either changed, revoked or renounced. Additionally, any attacker can pause or unpause any contract that expose `impl_pauser_for` macro.

Proof of Concept:

1. Attacker identifies admin address with DEFAULT_ADMIN_ROLE
2. Attacker calls: grant_role(admin_address, attacker_address, MINTER_ROLE)
3. Function only checks if admin_address HAS the role (it does)
4. Function NEVER verifies admin_address signed the transaction
5. Attacker now has MINTER_ROLE without any authorization

---

1. Place the test in `access-control/src/test.rs`

```rust
#[test]
fn test_grant_role_missing_auth_vulnerability() {
    let env = Env::default();

    let user1 = Address::from_str(
        &env,
        "GABE6NH6YU2TQWQGKPHBQ52F5YTT5LZCML6OSK7FTE4Q2BXI3D4KGCLL",
    );
    let user2 = Address::from_str(
        &env,
        "GAIH3ULLFQ4DGSECF2AR555KZ4KNDGEKN4AFI4SU2M7B43MGK3QJZNSR",
    );

    // Mock auth only for initialization
    env.mock_all_auths();
    let contract_id = env.register(Dummy, (user1.clone(),));
    let client = DummyClient::new(&env, &contract_id);

    let minter_role: String = String::from_str(&env, "MINTER_ROLE");

    env.mock_auths(&[]);

    client.grant_role(&user1, &user2, &minter_role);
```

```
    // Role successfully granted without authorization
    assert_eq!(client.has_role(&user2, &minter_role), true);
}
```

2. Execute with

```
cd libraries/access-control
cargo test test_grant_role_missing_auth_vulnerability  --features test-contract --lib
```

Test should pass, demonstrating that user has granted himself minter role.

Recommended Mitigation: Add the missing `caller.require_auth` to the vulnerable functions.

```
 pub fn grant_role(env: &Env, caller: &Address, account: &Address, role: &String) {
+       caller.require_auth();
```

STBL: Fixed in commit fa2f7bf.

Cyfrin: Verified. All the external functions in the `Pauser` and `AccessControl` libraries use `require_auth`, which prevents impersonation and role takeover.

7.1.2 Yield distribution blocked due to wrong `assert_issuer` usage

Description: Only `asset-vault` invokes `yield-distributor::distribute_reward`. However distributor has the `assert_issuer` authentication function, which allows only the `asset-issuer` to distribute.

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/asset-yield-distributor/src/lib.rs#L276

```
fn distribute_reward(env: Env, caller: Address, reward: u128) {
    caller.require_auth();
    let asset: AssetDefinition = Self::assert_issuer(&env, &caller);
```

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/asset-yield-distributor/src/lib.rs#L207

```
fn assert_issuer(env: &Env, caller: &Address) -> AssetDefinition {
    let asset_data: AssetDefinition = Self::fetch_asset_data(env);
    if !asset_data.is_issuer(caller) {
        panic!("STBL_Asset_InvalidIssuer");
    }
    asset_data
}
```

In the `register` contract, we can clearly see these 2 addresses (vault and issuer) won't be the same:

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/register/src/lib.rs#L345-L347

```
pub fn setup_asset(
    env: Env,
    caller: Address,
    id: u64,
    // contract_addr, issuance_addr, dist_addr, vault_addr, oracle,
    addresses: Vec<Address>,
        ...MORE ARGS
) {
    ...MORE CODE
    let contract_addr: Address = addresses.get(0).clone().unwrap();
    let issuance_addr: Address = addresses.get(1).clone().unwrap();
    let dist_addr: Address = addresses.get(2).clone().unwrap();
    let vault_addr: Address = addresses.get(3).clone().unwrap();
```

```
    let oracle: Address = addresses.get(4).clone().unwrap();

    asset.token = Some(contract_addr);
    asset.issuer = Some(issuance_addr);
    asset.reward_distributor = Some(dist_addr);
    asset.vault = Some(vault_addr);
```

Impact:

1. **Users cannot receive yield rewards** - The entire yield distribution mechanism is broken. Even when the underlying asset appreciates in value, rewards can never be distributed to stakers.
2. **Yield accumulates but is locked** - The vault calculates yield correctly (line 319-324 in vault), but the call to `distribute_reward` always fails, so:
   - `vault.yield_fees` never gets updated
   - `vault.asset_deposit_net` never gets reduced
   - The reward index in the distributor never increases
   - Stakers' earned rewards remain at 0
3. **Protocol is fundamentally non-functional** - Yield distribution is a core feature. Without it, users have no incentive to deposit/stake since they'll never earn rewards.
4. **No workaround exists** - The only caller of `distribute_reward` is the vault (line 333), and it always passes its own address. There's no alternative path to distribute rewards.

Proof of Concept:

1. in `integration-tests/tests` create new file `test_distribute_yield_blocked.rs`
2. place the following setup and test

```
use asset_issuer::AssetIssuerClient;
use integration_tests::TestIntegratedSystem;
use soroban_sdk::{testutils::Ledger, vec, Bytes, String, Vec};

use stbl_core_interfaces::{
    oracle::{AssetOracleClient, ExternalOracleClient},
    stbl_token::StblTokenClient,
    vault::VaultClient,
};

/// BUG: distribute_yield permanently blocked due to wrong caller check
///
/// Flow:
/// 1. AssetVault::distribute_yield (line 333) calls:
///    distributor_client.distribute_reward(&env.current_contract_address(), &yield_asset)
///    -> passes VAULT address as caller
///
/// 2. YieldDistributor::distribute_reward (line 278) calls:
///    Self::assert_issuer(&env, &caller)
///    -> checks if caller == asset.issuer
///
/// 3. Since vault != issuer, panics with "STBL_Asset_InvalidIssuer"
///
/// The is_vault check on line 287 is unreachable.
#[test]
#[should_panic(expected = "STBL_Asset_InvalidIssuer")]
fn test_distribute_yield_blocked_vault_not_issuer() {
    let system = TestIntegratedSystem::deploy();
    let env = &system.env;

    let register_client = register::StblRegisterClient::new(env, &system.registry);
```

```rust
// Deploy underlying token
let token = env.register(stbl_token::StblToken, (system.admin.clone(),));
let token_client = StblTokenClient::new(env, &token);
system.grant_minting_access(&token, &system.admin);

// Add asset
let asset_name = String::from_str(env, "TestAsset");
let description = String::from_str(env, "Test");
let asset_id = register_client.add_asset(&system.admin, &asset_name, &description, &1, &1, &true);

// Deploy oracle with initial price $1.00
let rwa_oracle = env.register(rwa_oracle::RwaOracle, (system.admin.clone(),));
let rwa_oracle_client = ExternalOracleClient::new(env, &rwa_oracle);
rwa_oracle_client.set_price_data(&1_000_000_000, &env.ledger().timestamp());
let oracle = env.register(
    asset_oracle::AssetOracle,
    (system.admin.clone(), rwa_oracle.clone()),
);
let oracle_client = AssetOracleClient::new(env, &oracle);
oracle_client.set_price_decimals(&system.admin, &9);
oracle_client.set_price_threshold(&system.admin, &60);

// Deploy issuer, vault, distributor
let issuer = env.register(
    asset_issuer::AssetIssuer,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let vault = env.register(
    asset_vault::AssetVault,
    (system.admin.clone(), system.registry.clone(), asset_id),
);
let distributor = env.register(
    asset_yield_distributor::AssetYieldDistributor,
    (system.admin.clone(), system.registry.clone(), asset_id),
);

// Setup asset with yield_duration = 1 second
let fees: Vec<u128> = vec![env, 0, 0, 10_000_000, 0]; // 1% yield fee
register_client.setup_asset(
    &system.admin,
    &asset_id,
    &vec![
        env,
        token.clone(),
        issuer.clone(),
        distributor.clone(),
        vault.clone(),
        oracle.clone(),
    ],
    &10,
    &u128::pow(10, 24),
    &fees,
    &7200,
    &1, // yield_duration = 1 second
    &Bytes::new(env),
);

// Mint tokens and deposit
let deposit_amount: u128 = 1_000_000_000_000;
token_client.mint(&system.admin, &system.alice, &deposit_amount);
token_client.approve(&system.alice, &vault, &(deposit_amount as i128), &1000);
```

```
    let issuer_client = AssetIssuerClient::new(env, &issuer);
    issuer_client.deposit(&system.alice, &deposit_amount);

    // Advance time past yield_duration
    env.ledger().with_mut(|li| {
        li.timestamp += 10;
    });

    // Increase oracle price to $1.10 to create yield
    rwa_oracle_client.set_price_data(&1_100_000_000, &env.ledger().timestamp());

    // Call distribute_yield - THIS WILL PANIC with "STBL_Asset_InvalidIssuer"
    // Because vault calls distributor.distribute_reward(&vault_address, ...)
    // But distributor checks assert_issuer(caller) which requires caller == issuer
    // vault != issuer, so it panics before reaching the is_vault check
    let vault_client = VaultClient::new(env, &vault);
    vault_client.distribute_yield(&system.admin);
}
```

3. Execute with `cargo test --package integration-tests --test test_distribute_yield_blocked`
4. Test should pass successfully due to the `should_panic` attribute with `STBL_Asset_InvalidIssuer` message

Recommended Mitigation: Leave only `asset.is_vault(&caller)` as auth in `distribute_reward`.

STBL: Fixed in commit b919058.

Cyfrin: Verified. `distribute_reward` now uses the correct function to fetch the asset data, without enforcing the caller being the issuer.

7.2 High Risk

7.2.1 Incorrect storage Key in `increase_nft_counter` causes NFT ownership overwrites

Description: `increase_nft_counter` in `yld-token` increases `DataKey::Version` instead of `DataKey::NftCtr`.

If we take a look at the function, we'll see the wrong key is being used:

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/yld-token/src/lib.rs#L58

```
fn increase_nft_counter(env: &Env) -> u32 {
  let key = DataKey::Version;//@audit WRONG key
  let current_nft_counter = get_nft_counter(env);
  env.storage()
      .persistent()
      .set(&key, &(current_nft_counter + 1));
  touch_persistent(env, &key);
  current_nft_counter + 1
}
```

Whereas in the `get_nft_counter`, `NftCtr` is being used, and it's the correct one.

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/yld-token/src/lib.rs#L46

```
fn get_nft_counter(env: &Env) -> u32 {
    let key = DataKey::NftCtr;
    match env.storage().persistent().get(&key) {
        Some(value) => {
            touch_persistent(env, &key);
            value
        }
        None => 0,
    }
}
```

Impact: The counter won't increase, and all the mints will use the same NFT ID - 1.

Furthermore, since OZ's mint implementation doesn't prevent reusing tokens, the same token, with id 1, will be minted to each user. This will overwrite the owner and seize it from the previous one.

https://github.com/OpenZeppelin/stellar-contracts/blob/82f97111f9568d32653dd5b3563baf5b73509c89/packages/tokens/src/non_fungible/storage.rs#L709-L717

```
/// **IMPORTANT**: This function does NOT verify whether the provided
    /// `token_id` already exists. It is the developer's responsibility to
    /// ensure `token_id` uniqueness before passing it to this function. The
    /// strategy for generating `token_id`s varies by project and must be
    /// implemented accordingly.
    pub fn mint(e: &Env, to: &Address, token_id: u32) {
        Base::update(e, None, Some(to), token_id);
        emit_mint(e, to, token_id);
    }
```

The counter stored under NftCtr never increments (remains 0)

1. `get_nft_counter` always returns 0, so every mint uses token ID 1.
2. Each new mint overwrites the previous owner of token ID 1.
3. Users lose their NFTs when subsequent users mint.

Recommended Mitigation: In `increase_nft_counter` replace with the correct key.

```
fn increase_nft_counter(env: &Env) -> u32 {
-   let key = DataKey::Version;
+   let key = DataKey::NftCtr;
    let current_nft_counter = get_nft_counter(env);
    env.storage()
        .persistent()
        .set(&key, &(current_nft_counter + 1));
    touch_persistent(env, &key);
    current_nft_counter + 1
}
```

STBL: Fixed in commit 1e61536.

Cyfrin: Verified. Correct storage key is being used in `increase_nft_counter`.

7.2.2 Missing Decimal Normalization Results in Incorrect USST Minting Amounts

Description: When a user deposits an RWA asset, the protocol calculates `stable_value_net` in `AssetIssuer::generate_metadata()` and mints this amount as USST in `StblCore::put()`:

```
// core/src/lib.rs:164-165
let _: () =
    usst_token_client.mint(&env.current_contract_address(), &to, &meta.stable_value_net);
```

The `stable_value_net` is derived from `stable_value_gross` which is calculated in the oracle:

```
// asset-oracle/src/lib.rs:198-207
fn fetch_forward_price(env: Env, asset_amount: u128) -> u128 {
    let price_decimals = get_price_decimals(&env);
    let current_price = AssetOracle::fetch_price(env.clone());

    let asset_value_with_decimals =
        U256::from_u128(&env, current_price).mul(&U256::from_u128(&env, asset_amount));
    let value =
        asset_value_with_decimals.div(&U256::from_u128(&env, u128::pow(10, price_decimals)));
    value.to_u128().unwrap()
}
```

The formula is: `stable_value = (asset_amount * price) / 10^price_decimals`

The problem is that `asset_amount` comes in the decimals of the underlying asset (which varies: 6 for USDC, 8 for WBTC, 18 for WETH, etc.), but USST always has 18 decimals. There is no normalization step to convert the result to 18 decimals before minting.

The code even contains a TODO comment acknowledging this missing functionality:

```
// asset-issuer/src/lib.rs:383-391
//  Normalize asset value to 18 decimals
// You already ported DecimalConverter; use that here.
// ...
// For now, if you want to keep it simple, you can just:
meta.asset_value = asset_value;
```

Impact: Consider a deposit of 100 USDC (6 decimals) worth $100, with a Chainlink-style price feed (8 decimals):

```
asset_amount = 100 * 10^6 = 100_000_000
price = 1 * 10^8 = 100_000_000 (representing $1.00)
price_decimals = 8

stable_value_gross = (100_000_000 * 100_000_000) / 10^8 = 100_000_000
```

The protocol mints `100_000_000` USST. But USST has 18 decimals, so this represents `100_000_000 / 10^18 = 0.0000000001` USST instead of the expected 100 USST (`100 * 10^18`).

The user receives mass value reduction of 10^10x for assets with 6 decimals, and varying degrees of loss/gain depending on the decimal mismatch. This completely breaks the protocol's core functionality of minting stablecoins proportional to deposited asset value.

Recommended Mitigation: Implement decimal normalization in `generate_metadata()` to convert `stable_value_gross` to 18 decimals before calculating fees and `stable_value_net`:

```
fn generate_metadata(env: &Env, asset: &AssetDefinition, asset_value: u128) -> YLDMetadata {
    // ...

    // Get asset token decimals
    let token = asset.token.clone().unwrap();
    let token_client = TokenClient::new(env, &token);
    let asset_decimals = token_client.decimals();

    // Normalize to 18 decimals
    let normalized_asset_value = if asset_decimals < 18 {
        asset_value * 10u128.pow(18 - asset_decimals)
    } else if asset_decimals > 18 {
        asset_value / 10u128.pow(asset_decimals - 18)
    } else {
        asset_value
    };

    meta.asset_value = normalized_asset_value;
    // ... continue with oracle pricing using normalized value
}
```

STBL: Fixed in commit 39a9e3b4.

Cyfrin: Verified.

7.2.3 YLD Token Burn Uses `asset_id` Instead of `token_id` for Ownership Verification

Description: The `YldToken::burn()` function contains a critical bug where it checks ownership using `asset_id` instead of `token_id`:

```
// yld-token/src/lib.rs:191-210
pub fn burn(env: Env, caller: Address, from: Address, token_id: u32) {
    AccessControl::require_role(&env, &caller, &String::from_str(&env, MINTER_ROLE));
    Pauser::when_not_paused(&env);

    let mut yld_metadata: YLDMetadata = is_yld_transferable(&env, token_id);
    if Base::owner_of(&env, yld_metadata.asset_id as u32) != from {
        panic!("STBL_YLD_NotOwner");
    }

    Base::burn_from(&env, &caller, &from, token_id);
    // ...
}
```

The function retrieves metadata for the correct `token_id`, but then checks if `from` owns a *different* NFT - one with ID equal to `yld_metadata.asset_id`. Since `asset_id` represents the type of underlying RWA asset (e.g., 1 for USDC vault, 2 for WBTC vault), not the NFT identifier, this check is semantically incorrect.

The burn is called from `StblCore::exit()` during user withdrawals:

```
// core/src/lib.rs:220-222
let yld = get_yld_address(&env);
let yld_token_client = YldTokenClient::new(&env, &yld);
```

```
let _: () = yld_token_client.burn(&env.current_contract_address(), &from, &token_id);
```

Which in turn is called from `AssetIssuer::internal_withdraw()`:

```
// asset-issuer/src/lib.rs:356-364
let core_addr = reg.fetch_core();
let core = StblCoreClient::new(env, &core_addr);
core.exit(
    &env.current_contract_address(),
    &get_asset_id(env),
    &owner.clone(),
    &token_id,
    &yld_metadata.stable_value_net,
);
```

Impact: The ownership check `Base::owner_of(&env, yld_metadata.asset_id as u32) != from` will almost always fail because:

1. If `asset_id = 1` (first registered asset), the check asks "does `from` own NFT №1?"
2. But `from` might own NFT №500 (their actual `token_id`)
3. NFT №1 belongs to whoever made the very first deposit to asset type 1

This means virtually all withdrawal attempts will revert with `STBL_YLD_NotOwner`, permanently locking user funds in the protocol. The only exception is the rare case where a user happens to own both:

- The NFT they're trying to withdraw (`token_id`)
- The NFT whose ID equals the asset type number (`asset_id`)

For any asset type with `asset_id > total_minted_nfts`, the check will panic because that NFT doesn't exist at all.

Recommended Mitigation: Replace `asset_id` with `token_id` in the ownership check:

```
pub fn burn(env: Env, caller: Address, from: Address, token_id: u32) {
    AccessControl::require_role(&env, &caller, &String::from_str(&env, MINTER_ROLE));
    Pauser::when_not_paused(&env);

    let mut yld_metadata: YLDMetadata = is_yld_transferable(&env, token_id);
    if Base::owner_of(&env, token_id) != from {
        panic!("STBL_YLD_NotOwner");
    }

    Base::burn_from(&env, &caller, &from, token_id);
    // ...
}
```

STBL: Fixed in commit a5506b.

Cyfrin: Verified. Ownership check is done on the `token_id` now.

7.2.4 Inverted YLD Validity Check Blocks All Reward Claims

Description: The `AssetYieldDistributor::fetch_yld_metadata()` function contains an inverted validity check that prevents all reward claims:

```
// asset-yield-distributor/src/lib.rs:230-241
fn fetch_yld_metadata(env: &Env, yld_token: &Address, id: u32) -> YLDMetadata {
    let yld_token_client = YldTokenClient::new(&env, yld_token);
    let metadata: Option<YLDMetadata> = yld_token_client.get_nft_data(&id);
    if metadata.is_none() {
        panic!("STBL_Asset_YldMetadatNotFound");
    }
```

```
    let metadata: YLDMetadata = metadata.unwrap();
    if metadata.is_valid() {
        panic!("STBL_YLD_Disabled");
    }
    metadata
}
```

The `is_valid()` method on `YLDMetadata` returns `true` when the NFT is in a valid, non-disabled state:

```
// stbl-core-types/src/asset_definition.rs:154-156
pub fn is_valid(&self) -> bool {
    self.is_disabled == false
}
```

The check `if metadata.is_valid() { panic!(...) }` causes the function to revert when the YLD NFT is valid and operational. The intended behavior is the opposite: revert when the NFT is disabled, allow when valid.

This function is called from `claim()`:

```
// asset-yield-distributor/src/lib.rs:377-405
fn claim(env: Env, caller: Address, id: u64) -> u128 {
    caller.require_auth();
    let asset: AssetDefinition = Self::fetch_asset_data(&env);

    let yld_token: Address = Self::fetch_yld_token(&env);

    let _metadata: YLDMetadata = Self::fetch_yld_metadata(&env, &yld_token, id as u32);
    // ... reward distribution logic
}
```

The `claim()` function is also called during the withdrawal process in `AssetIssuer::internal_withdraw()`:

```
// asset-issuer/src/lib.rs:336-340
let dist_addr = asset
    .reward_distributor
    .clone()
    .expect("reward_distributor not set");
let dist = YieldDistributorClient::new(env, &dist_addr);
dist.claim(&env.current_contract_address(), &(token_id as u64));
```

Impact: Every call to `claim()` will revert with `STBL_YLD_Disabled` for any valid, non-disabled YLD NFT. This has cascading effects:

1. Users cannot claim their accumulated yield rewards directly via `claim()`
2. User withdrawals via `AssetIssuer::withdraw()` will fail because `internal_withdraw()` calls `claim()` before processing the withdrawal. This means users cannot withdraw their deposited assets.
3. Treasury withdrawals via `withdraw_expired()` are also affected when the NFT is disabled, as it also calls `claim()`:

```
// asset-issuer/src/lib.rs:205-211
if yld_metadata.is_disabled {
    let dist_addr = asset
        .reward_distributor
        .clone()
        .expect("reward_distributor not set");
    let dist = YieldDistributorClient::new(&env, &dist_addr);
    dist.claim(&env.current_contract_address(), &(token_id as u64));
}
```

The result is that all user funds deposited into the protocol become permanently locked. Users cannot withdraw their RWA assets, and accumulated yield rewards are inaccessible.

Recommended Mitigation: Invert the condition to revert when the NFT is disabled:

```
fn fetch_yld_metadata(env: &Env, yld_token: &Address, id: u32) -> YLDMetadata {
    let yld_token_client = YldTokenClient::new(&env, yld_token);
    let metadata: Option<YLDMetadata> = yld_token_client.get_nft_data(&id);
    if metadata.is_none() {
        panic!("STBL_Asset_YldMetadatNotFound");
    }
    let metadata: YLDMetadata = metadata.unwrap();
    if !metadata.is_valid() {
        panic!("STBL_YLD_Disabled");
    }
    metadata
}
```

STBL: Fixed in commit 1845c1.

Cyfrin: Verified. Function uses the correct validity check, allowing reward claims.

7.2.5 Missing Token Approval Breaks Yield Distribution

Description: The `AssetVault::distribute_yield()` function calls `YieldDistributor::distribute_reward()`, which attempts to pull tokens from the Vault using `transfer_from`. However, the Vault never grants token approval to the YieldDistributor, causing all yield distributions to fail.

In the Vault, `distribute_yield()` calculates the yield and calls the distributor without any prior approval:

```
// asset-vault/src/lib.rs:326-329
let distributor = asset.reward_distributor.unwrap();
let distributor_client = YieldDistributorClient::new(&env, &distributor);

distributor_client.distribute_reward(&env.current_contract_address(), &yield_asset);
```

The YieldDistributor then attempts to transfer tokens from the Vault:

```
// asset-yield-distributor/src/lib.rs:293-299
let token_client = UsstTokenClient::new(&env, &asset.token.clone().unwrap());
let _: () = token_client.transfer_from(
    &env.current_contract_address(),  // spender = YieldDistributor
    &caller,                          // from = Vault
    &env.current_contract_address(),  // to = YieldDistributor
    &(reward as i128),
);
```

In Soroban's SEP-41 token standard, `transfer_from` requires the token owner (Vault) to have previously approved the spender (YieldDistributor). Since no approval is ever granted, the `transfer_from` call will always fail.

The Solidity implementation correctly handles this by having the Vault approve the YieldDistributor before calling `distributeReward`:

```
// STBL_PT1_Vault.sol:284-298
IERC20(AssetData.token).approve(
    AssetData.rewardDistributor,
    yieldAssetValue.convertFrom18Decimals(...)
);

iSTBL_PT1_AssetYieldDistributor(AssetData.rewardDistributor)
    .distributeReward(yieldAssetValue...);
```

Impact: All yield distribution attempts will revert. Users staking YLD tokens cannot receive any yield rewards, and accumulated yield remains locked in the Vault with no mechanism to distribute it.

Recommended Mitigation: Add approval in the Vault before calling `distribute_reward`:

```
let token = asset.token.unwrap();
let token_client = UsstTokenClient::new(&env, &token);

// Approve YieldDistributor to spend yield tokens
token_client.approve(
    &env.current_contract_address(),
    &distributor,
    &(yield_asset as i128),
    &(env.ledger().max_live_until_ledger()),
);

distributor_client.distribute_reward(&env.current_contract_address(), &yield_asset);
```

Alternatively, change to a push model where the Vault transfers tokens directly to the YieldDistributor before notifying it, eliminating the need for approval management.

STBL: Fixed in commits 13a2fd2, a0886a8.

Cyfrin: Verified.

7.3 Medium Risk

7.3.1 PT Vault distributeYield Missing Treasury Authorization

Description: In the Solidity implementation, `distributeYield()` has different access control depending on the asset type:

PT1 Vault — only treasury can call:

```
// STBL_PT1_Vault.sol:264-269
function distributeYield() external {
    AssetDefinition memory AssetData = registry.fetchAssetData(assetID);
    uint256 differentialUSD = iCalculatePriceDifferentiation();

    if (msg.sender != registry.fetchTreasury())
        revert STBL_UnauthorizedIssuer();
    // ...
}
```

LT1 Vault — permissionless:

```
// STBL_LT1_Vault.sol:261-264
function distributeYield() external {
    AssetDefinition memory AssetData = registry.fetchAssetData(assetID);
    uint256 differentialUSD = iCalculatePriceDifferentiation();
    // no treasury check
    // ...
}
```

In the Soroban implementation, there is a single `AssetVault` contract that handles both PT and LT assets, but `distribute_yield()` is permissionless for all asset types:

```
// asset-vault/src/lib.rs:299-340
fn distribute_yield(env: Env, caller: Address) {
    caller.require_auth();

    let asset: AssetDefinition = Self::fetch_asset_data(&env);
    // no check for contract_type == PT
    // no check for caller == treasury
    // ...
}
```

The `AssetDefinition` struct includes a `contract_type` field that distinguishes between LT and PT assets, but this is not used to enforce the treasury-only restriction for PT assets.

Impact: For PT-type assets, anyone can trigger yield distribution instead of only the treasury. This deviates from the intended access control model where PT yield distribution is a privileged operation controlled by the protocol treasury.

Recommended Mitigation: Add a check for PT assets to require treasury authorization:

```
fn distribute_yield(env: Env, caller: Address) {
    caller.require_auth();

    let asset: AssetDefinition = Self::fetch_asset_data(&env);

    // PT assets require treasury authorization
    if asset.contract_type == ContractType::PT {
        let registry = get_registry(&env);
        let reg_client = StblRegisterClient::new(&env, &registry);
        let treasury = reg_client.fetch_treasury();
        if caller != treasury {
            panic!("STBL_UnauthorizedCaller");
        }
```

```
    }

    // ... rest of function
}
```

STBL: Fixed in commit 69b8be06. Added treasury check for both PT and LT.

Cyfrin: Verified. Now both PT and LT assets vault `distributeYield` function can be called only by treasury.

7.3.2 Persistent Storage TTL Expiry Causes Security Bypass and State Corruption

Description: Multiple critical variables across the codebase use persistent storage with a TTL of ~150 days. Since there's no getter function to extend the ledger for some of these, after ~150 days they will expire.

This happens because `unwrap_or(false)` or `None => false` patterns return default values after being archived, silently changing contract behavior.

Affected Variables:

Location	DataKey	Default After Expiry
libraries/access-control/src/lib.rs:35	Init	false
libraries/pauser/src/lib.rs:45	Paused	false
contracts/usst-token/src/lib.rs:57	Blacklisted(Address)	false
contracts/asset-oracle/src/lib.rs:34	Enabled	false
contracts/yld-token/src/lib.rs:53	NftCtr	0
contracts/register/src/lib.rs:62	AssetCounter	0

1. Pauser Library

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/libraries/pauser/src/lib.rs#L45

```
pub fn is_paused(env: &Env) -> bool {
  let key = get_key();
  touch_persistent(&env, &key);
  env.storage().persistent().get(&key).unwrap_or(false)
}
```

If the contract is paused but no one interacts with `is_paused()` for ~150 days, the storage entry expires and `unwrap_or(false)` returns `false`. A paused contract will appear unpaused.

2. Blacklist in USST Token

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/usst-token/src/lib.rs#L50-L58

```
fn is_blacklisted(env: &Env, account: &Address) -> bool {
  let key = DataKey::Blacklisted(account.clone());
  match env.storage().persistent().get(&key) {
      Some(value) => {
          touch_persistent(&env, &key);
          value
      }
      None => false,
  }
}
```

If a blacklisted address doesn't interact with the contract for ~150 days, the blacklist entry expires and `None => false` treats them as non-blacklisted. A sanctioned/malicious address can bypass the blacklist.

3. Oracle Enabled Flag

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/asset-oracle/src/lib.rs#L27-L35

```
fn is_enabled(env: &Env) -> bool {
    let key = DataKey::Enabled;
    match env.storage().persistent().get(&key) {
        Some(value) => {
            touch_persistent(&env, &key);
            value
        }
        None => false,
    }
}
```

If the oracle isn't queried for ~150 days, the enabled flag expires and `None => false` disables it. An enabled oracle will appear disabled.

4. Counter Variables

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/yld-token/src/lib.rs#L46-L54

```
fn get_nft_counter(env: &Env) -> u32 {
    let key = DataKey::NftCtr;
    match env.storage().persistent().get(&key) {
        Some(value) => {
            touch_persistent(env, &key);
            value
        }
        None => 0,
    }
}
```

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/register/src/lib.rs#L55-L63

```
fn get_asset_counter(env: &Env) -> u64 {
  let key = DataKey::AssetCounter;
  match env.storage().persistent().get(&key) {
      Some(value) => {
          touch_persistent(&env, &key);
          value
      }
      None => 0,
  }
}
```

If no new NFTs or assets are created for ~150 days, the counters expire and reset to `0`. New items will have colliding IDs with existing ones.

Impact: | Variable | Impact | Severity | | --- | --- | --- | | `Paused` | Paused contract becomes operational, bypassing emergency stop | High | | `Blacklisted` | Sanctioned addresses can transfer tokens again | High | | `Init` | Reinitialization possible if exposed publicly | Low | | `Enabled` | Oracle appears disabled, breaking price feeds | Medium | | `NftCtr` | NFT ID collisions, overwriting existing NFTs | Medium | | `AssetCounter` | Asset ID collisions, overwriting existing assets | Medium |

Proof of Concept: Test only for the `asset-control` to demonstrate the side effect, other variables use the same patterns.

1. Add to `access-control/test-contract.rs`

```rust
pub fn reinitialize(env: Env, caller: Address) {
    caller.require_auth();
    AccessControl::init_access_control(&env, &caller);
}
```

2. Add the POC to test file

```rust
#[test]
fn test_reinitialization_after_storage_expiry() {
    use soroban_sdk::testutils::Ledger;

    let env = Env::default();
    env.mock_all_auths();

    // Initialize contract with user1 as admin
    let user1 = Address::from_str(
        &env,
        "GABE6NH6YU2TQWQGKPHBQ52F5YTT5LZCML6OSK7FTE4Q2BXI3D4KGCLL",
    );
    let contract_id = env.register(Dummy, (user1.clone(),));
    let client = DummyClient::new(&env, &contract_id);

    let default_admin_role: String = soroban_sdk::String::from_str(&env, "DEFAULT_ADMIN_ROLE");
    assert!(client.has_role(&user1, &default_admin_role));

    // Advance ledger beyond PERSISTENT_EXTEND_TO (2,500,000 ledgers)
    let ledger_sequence_before = env.ledger().sequence();
    env.ledger().with_mut(|li| {
        li.sequence_number = ledger_sequence_before + 2_600_000;
        li.timestamp += 2_600_000 * 5;
    });

    // Verify we passed the TTL threshold
    assert!(env.ledger().sequence() > ledger_sequence_before + 2_500_000);

    // Simulate storage expiry by manually removing the DataKey::Init entry
    // This is what would happen automatically in production after TTL expiry
    env.as_contract(&contract_id, || {
        use crate::DataKey;
        env.storage().persistent().remove(&DataKey::Init);
    });

    // Now an attacker (user2) can reinitialize and gain admin control
    let user2 = Address::from_str(
        &env,
        "GAIH3ULLFQ4DGSECF2AR555KZ4KNDGEKN4AFI4SU2M7B43MGK3QJZNSR",
    );

    // Attacker successfully reinitializes the contract
    client.reinitialize(&user2);

    // Verify the attacker now has admin privileges
    assert!(
        client.has_role(&user2, &default_admin_role),
        "Attacker gained admin control after storage expiry"
    );

    // Both users now have admin role - contract is compromised
    assert!(client.has_role(&user1, &default_admin_role));
    assert!(client.has_role(&user2, &default_admin_role));
}
```

3. Execute with

```
cd libraries/access-control
cargo test test_reinitialization_after_storage_expiry --features test-contract --lib
```

Recommended Mitigation: For variables that are critical for security or aren't touched frequently, consider using `instance` type of storage, as it lives as long as the contract lives, without risk of being archived.

https://developers.stellar.org/docs/build/guides/storage/use-instance

Variables that should migrate to instance storage:

- `DataKey::Init` (access-control)
- `DataKey::Paused` (pauser)
- `DataKey::Blacklisted` (usst-token)
- `DataKey::Enabled` (asset-oracle)
- `DataKey::NftCtr` (yld-token)
- `DataKey::AssetCounter` (register)

STBL: Fixed in commit c1d878d.

Cyfrin: Verified. Correct storage types are being used, with consistent TTL extensions.

7.3.3 Withdraw Fee Incorrectly Included in Staking Balance

Description: The withdrawal fee is calculated at deposit time but is not deducted from `stable_value_net`:

```rust
// asset-issuer/src/lib.rs:419-420
meta.withdraw_fee_amount =
    (meta.stable_value_gross * meta.fees.withdraw_fee as u128) / FEES_CONSTANT as u128;

// asset-issuer/src/lib.rs:426-427
meta.stable_value_net = meta.stable_value_gross
    - (meta.deposit_fee_amount + meta.haircut_amount + meta.insurance_fee_amount);
// withdraw_fee_amount is NOT subtracted
```

Unlike other fees (deposit, haircut, insurance), the withdrawal fee remains included in `stable_value_net`. This value is then used throughout the protocol in ways that create accounting inconsistencies.

Impact: 1. USST is minted for value that belongs to the protocol

`Core::put` mints USST equal to `stable_value_net`:

```rust
// core/src/lib.rs:164-165
usst_token_client.mint(&env.current_contract_address(), &to, &meta.stable_value_net);
```

Since `stable_value_net` includes the withdrawal fee, users receive USST tokens representing value they will never be able to redeem. For a 10,000 USD deposit with 5% withdrawal fee, the user receives 500 USST that are effectively unbacked from their perspective — these funds belong to the protocol.

2. Withdrawal fee generates yield for users instead of protocol

The staking balance includes the withdrawal fee portion:

```rust
// asset-issuer/src/lib.rs:278
let total_stake = yld_metadata.stable_value_net + yld_metadata.haircut_amount;
dist.enable_staking(&env.current_contract_address(), &(token_id as u64), &total_stake);
```

This means yield accrues on value that will be taken as protocol fees. For a position with 500 USD withdrawal fee held for one year at 5% APY, the protocol loses ~25 USD in yield that should have accrued to itself or other stakers.

3. Yield distribution is skewed toward high-fee positions

Consider two identical deposits where only withdrawal fee differs:

	User A (0% fee)	User B (5% fee)
Actual redeemable value	10,000 USD	9,500 USD
Staking balance	10,000	10,000
Yield share	50%	50%

User B receives the same yield share as User A despite having 500 USD less actual value. This effectively transfers yield from users with lower withdrawal fees to users with higher withdrawal fees.

If withdrawal fees vary by asset type, users depositing high-fee assets systematically receive disproportionate yield at the expense of users depositing low-fee assets.

4. Protocol overcollateralization goal is undermined

The protocol deducts haircut specifically to maintain overcollateralization — the collateral backing exceeds the USST minted. However, by not deducting withdrawal fees, the protocol mints more USST than users can actually redeem, working against this goal.

Recommended Mitigation: If the protocol intends for withdrawal fees to be recalculated at withdrawal time (e.g., fees can change based on market conditions):

1. Remove `withdraw_fee_amount` calculation from deposit flow entirely — do not store it in `YLDMetadata`
2. Calculate withdrawal fee dynamically in `Vault::withdraw_erc20` by reading current fee rate from the registry
3. Deduct a placeholder or expected fee from `stable_value_net` at deposit for USST minting and staking purposes, or accept the accounting implications as a design tradeoff

STBL: Fixed in commit e4c5978. Withdrawal fees are calculated at the time of withdrawal. The fee was static; however, we updated the logic to make it dynamic, where the fee is derived from the latest configuration during withdrawal. This change allows the underlying amount to be utilized for yield generation and USST minting.

Cyfrin: Verified. Now withdrawal fees are calculated at withdrawal time. The fact that withdrawal fees are used by the user until the withdrawal is intended design.

7.4 Low Risk

7.4.1 Oracle Initialized with Zero Price Decimals Breaks Price Calculations

Description: The `AssetOracle` constructor initializes `price_decimals` to 0 while simultaneously marking the oracle as enabled:

```
pub fn __constructor(env: Env, caller: Address, rwa_oracle: Address) {
    // ...
    update_flag(&env, true);          // oracle is ENABLED
    set_price_decimals(&env, &0);     // but decimals = 0
    set_price_threshold(&env, &0);
}
```

In `fetch_forward_price`, this value is used as a divisor to normalize the price:

```
fn fetch_forward_price(env: Env, asset_amount: u128) -> u128 {
    let price_decimals = get_price_decimals(&env);
    let current_price = AssetOracle::fetch_price(env.clone());

    let asset_value_with_decimals =
        U256::from_u128(&env, current_price).mul(&U256::from_u128(&env, asset_amount));
    let value =
        asset_value_with_decimals.div(&U256::from_u128(&env, u128::pow(10, price_decimals)));
    // When price_decimals = 0: divides by 10^0 = 1, no normalization occurs
}
```

With `price_decimals = 0`, the formula becomes `stable_value = price * asset_amount / 1 = price * asset_amount`. This produces values orders of magnitude larger than intended, since oracle prices typically have 8-18 decimals that should be normalized out.

Impact: The oracle is immediately usable after deployment (`enabled = true`) but will return incorrect values until admin manually calls `set_price_decimals()`. There is no validation or warning that decimals must be configured first.

If the oracle is used before configuration, `stable_value_gross` will be astronomically inflated. For example, with a Chainlink-style price of `2000 * 10^8` (representing $2000 with 8 decimals) and 100 tokens with 18 decimals, the expected USD value should be $200,000. Instead, with `price_decimals = 0`, the result would be `2000 * 10^8 * 100 * 10^18 = 2 * 10^31` — a value 10^8 times larger than intended.

This cascades through the entire protocol: USST minting amounts become wrong, all fee calculations based on `stable_value_gross` are incorrect, and deposit limit checks in the registry may overflow or incorrectly pass due to the inflated values.

Recommended Mitigation: Either require `price_decimals` as a constructor parameter, or initialize the oracle as disabled until decimals are configured:

```
pub fn __constructor(env: Env, caller: Address, rwa_oracle: Address, price_decimals: u32) {
    // ...
    update_flag(&env, true);
    set_price_decimals(&env, &price_decimals);
}
```

STBL: Fixed in commit 20694d03. Now, Taking price decimal and threshold in constructor arguments.

Cyfrin: Verified.

7.4.2 No Slippage Protection on Deposit and Withdraw Operations

Description: The `AssetIssuer::deposit()` and `AssetIssuer::withdraw()` functions do not accept any slippage protection parameters:

```
pub fn deposit(env: Env, caller: Address, asset_value: u128) -> u32 {
```

```
    caller.require_auth();
    Self::internal_deposit(&env, asset_value, caller.clone(), caller)
}

pub fn withdraw(env: Env, caller: Address, token_id: u32) {
    caller.require_auth();
    Self::internal_withdraw(&env, token_id, caller);
}
```

The final amount of USST minted on deposit and the amount of underlying asset returned on withdrawal depend on two external factors that can change between transaction submission and execution:

1. Registry Fee Parameters

In `generate_metadata()`, all fee parameters are fetched from the current Registry state at execution time:

```
meta.fees.deposit_fee = asset.deposit_fees;
meta.fees.withdraw_fee = asset.withdraw_fees;
meta.fees.hair_cut = asset.cut;
meta.fees.insurance_fee = asset.insurance_fees;
```

These parameters directly affect the net USST minted:

```
meta.stable_value_net = meta.stable_value_gross
    - (meta.deposit_fee_amount + meta.haircut_amount + meta.insurance_fee_amount);
```

Admins can modify these via `set_fees()`, `set_cut()`, and `set_durations()` at any time.

2. Oracle Price

The oracle price is fetched during deposit to calculate `stable_value_gross`:

```
// asset-issuer/src/lib.rs:406
meta.stable_value_gross = oracle.fetch_forward_price(&meta.asset_value);
```

This value determines how much USST the user receives. Additionally, the haircut is converted back to asset terms using the oracle:

```
// asset-issuer/src/lib.rs:420
meta.haircut_amount_asset_value = oracle.fetch_inverse_price(&meta.haircut_amount);
```

On withdrawal, the vault uses the oracle to calculate the actual asset amount to transfer:

```
// asset-vault/src/lib.rs:258-260
let withdraw_asset_value = oracle_client.fetch_inverse_price(
    &(metadata.stable_value_net + metadata.haircut_amount - metadata.withdraw_fee_amount),
);
```

Oracle prices naturally fluctuate between transaction submission and execution. More critically, admin-controlled parameters like `price_decimals` can be changed via `set_price_decimals()`, drastically affecting the conversion rate.

Impact: Users have no protection against either source of value change:

For example: A user depositing 1000 USDC expecting to receive ~990 USST (assuming 1% fees) could receive significantly less if fees are increased or if oracle price drops between submission and execution.

Recommended Mitigation: Add slippage protection parameters to both deposit and withdraw functions:

```
pub fn deposit(env: Env, caller: Address, asset_value: u128, min_usst_out: u128) -> u32 {
    caller.require_auth();
    let token_id = Self::internal_deposit(&env, asset_value, caller.clone(), caller);

    let yld_metadata = /* fetch metadata for token_id */;
```

```
    if yld_metadata.stable_value_net < min_usst_out {
        panic!("STBL_SlippageExceeded");
    }
    token_id
}

pub fn withdraw(env: Env, caller: Address, token_id: u32, min_asset_out: u128) {
    caller.require_auth();
    let asset_received = Self::internal_withdraw(&env, token_id, caller);
    if asset_received < min_asset_out {
        panic!("STBL_SlippageExceeded");
    }
}
```

STBL: Fixed in commit 096a05.

Cyfrin: Verified.

7.4.3 No Validation That Asset Token Supports Required Interface

Description: The `AssetYieldDistributor` and `AssetVault` contracts interact with asset tokens via `UsstTokenClient`:

```
// asset-yield-distributor/src/lib.rs:293
let token_client = UsstTokenClient::new(&env, &asset.token.clone().unwrap());
let _: () = token_client.transfer_from(...);
```

```
// asset-vault/src/lib.rs:193
let token_client = UsstTokenClient::new(&env, &token);
token_client.transfer_from(...);
```

When registering an asset via `setup_asset()`, the token address is accepted without any validation that it implements the required interface:

```
// register/src/lib.rs:352-358
let contract_addr: Address = addresses.get(0).clone().unwrap();
// ...
asset.token = Some(contract_addr);  // no interface check
```

In Solidity, `safeTransferFrom` gracefully handles incompatible tokens. In Soroban, calling methods on a contract that doesn't implement them will fail with a less descriptive error at runtime.

Impact: If an asset is registered with a token that doesn't implement the expected interface (e.g., a standard SEP-41 token without USST-specific functions), operations like `deposit_erc20`, `withdraw_erc20`, and `distribute_yield` etc will fail at runtime with unclear errors.

Recommended Mitigation: Document that only tokens implementing the full `UsstTokenTrait` interface are supported. Consider adding validation during `setup_asset()` to verify the token contract responds to expected methods before allowing registration.

STBL: Acknowledged. Additional safety and validation mechanisms will be introduced in the token setup function. Furthermore, rather than relying on a standard trait client, distinct traits will be defined for each token type to improve clarity and security.

7.4.4 Emergency Withdraw Does Not Reset Vault Accounting State

Description: The `emergency_withdraw()` function transfers all tokens from the vault to treasury but does not reset the `VaultStruct` accounting fields:

```
// asset-vault/src/lib.rs:394-419
fn emergency_withdraw(env: Env, caller: Address) {
    caller.require_auth();
```

```
    // ...
    if asset.is_active() {
        panic!("STBL_AssetActive");
    }
    // ...
    token_client.transfer(&env.current_contract_address(), &treasury, &balance);

    // VaultStruct fields are NOT reset:
    // - asset_deposit_net, asset_deposit_gross, deposit_value_usd remain stale
    // - deposit_fees, withdraw_fees, yield_fees, insurance_fees remain stale
}
```

After `emergency_withdraw()` executes, the vault has zero token balance but `VaultStruct` still contains historical values for `asset_deposit_net`, `asset_deposit_gross`, `deposit_value_usd`, `cumulative_hair_cut_value`, and accumulated fees (`deposit_fees`, `withdraw_fees`, `yield_fees`, `insurance_fees`).

Additionally, `withdraw_fees()` is permissionless and can be called regardless of asset status. If a user calls `emergency_withdraw()` before protocol calls `withdraw_fees()`, the fee accounting fields will contain non-zero values but the tokens have already been transferred to treasury. A subsequent `withdraw_fees()` call will attempt to transfer fees that no longer exist in the vault, causing the transaction to revert.

The protocol cannot simply re-enable the asset via `enable_asset()` because the vault's accounting state is now inconsistent with its actual balance. Any deposit or withdrawal operations would produce incorrect calculations based on stale data. Since there is no `set_vault()` function in Register to point the asset to a fresh vault contract, the protocol must create an entirely new asset registration with new contracts.

However, existing YLD NFTs reference the old `asset_id`. If the protocol wants to honor these obligations, treasury must manually transfer funds to the old vault address (which still has corrupted accounting state) or coordinate off-chain settlements with affected users.

Any user can trigger this scenario by calling `emergency_withdraw()` while the asset is inactive, forcing the protocol into this operational complexity.

Impact: After emergency withdrawal, the protocol cannot reactivate the asset without deploying fresh vault contracts and re-registering under a new asset ID, while existing YLD NFT holders tied to the old asset ID require manual off-chain coordination for settlement.

Recommended Mitigation: Reset all `VaultStruct` fields in `emergency_withdraw()`:

```
fn emergency_withdraw(env: Env, caller: Address) {
    // ... existing logic ...

    token_client.transfer(&env.current_contract_address(), &treasury, &balance);

    set_vault_data(&env, &VaultStruct {
        deposit_fees: 0,
        withdraw_fees: 0,
        insurance_fees: 0,
        yield_fees: 0,
        cumulative_hair_cut_value: 0,
        deposit_value_usd: 0,
        asset_deposit_gross: 0,
        asset_deposit_net: 0,
    });
}
```

Alternatively, restrict `emergency_withdraw()` to treasury-only access to prevent users from triggering this state.

STBL: Fixed in commit 97b62f46. Not resetting the vault state for Accounting purposes and consistency with EVM code.

Cyfrin: Verified. Now `withdraw_fees` can be called only by treasury and `emergency_withdraw` can be called only with `EMERGENCY_ROLE` and after `withdraw_fees`.

7.4.5 `asset-oracle` is missing price validation

Description: `asset-oracle` doesn't validate the price returned by the RWA oracle. If the price source starts transmitting false values - 0 or invalid higher/lower prices that are not actual, this will go unnoticed by the protocol, and the price will be considered valid as long as it passes the validity threshold.

https://github.com/USD-Pi-Protocol/stellar-core-contracts/blob/main/contracts/asset-oracle/src/lib.rs#L190

```
fn fetch_price(env: Env) -> u128 {
    if !is_enabled(&env) {
        panic!("OracleError::OracleDisabled");
        // panic_with_error!(env, OracleError::OracleDisabled);
    }
    // Load external oracle
    let rwa_addr = get_rwa_oracle(&env);
    let rwa = ExternalOracleClient::new(&env, &rwa_addr);

    let (price, timestamp) = rwa.get_price_data();

    let threshold = get_price_threshold(&env);
    let now = env.ledger().timestamp();

    // Stale price check
    if timestamp + threshold <= now {
        panic!("OracleError::OracleStalePrice");
        // panic_with_error!(env, OracleError::OracleStalePrice);
    }

    price
}
```

Impact: If the external RWA oracle returns 0 (due to malfunction or compromise), `fetch_inverse_price` will panic due to division by zero at lib.rs:215, causing a denial of service for any functionality depending on inverse price calculations. Additionally, fetch_forward_price would return 0 for any asset amount, potentially causing incorrect valuations throughout the protocol. Abnormally high or low prices would also pass through unchecked, leading to incorrect forward and inverse price calculations.

Recommended Mitigation: Add a zero-price validation check after fetching the price from the oracle:

```
+ if price == 0 {
+    panic!("OracleError::OracleInvalidPrice");
+ }
```

Optionally, consider implementing a price deviation threshold to reject prices that change beyond a configurable percentage from the previous known price.

STBL: Fixed in commit f54cbc6.

Cyfrin: Verified.

7.4.6 Single Oracle Dependency Can Freeze User Funds

Description: The protocol relies on a single oracle per asset with no fallback mechanism. The `fetch_price()` function enforces two hard requirements that, if not met, will revert all price-dependent operations:

```
fn fetch_price(env: Env) -> u128 {
    if !is_enabled(&env) {
        panic!("OracleError::OracleDisabled");
    }

    let (price, timestamp) = rwa.get_price_data();
    let threshold = get_price_threshold(&env);
```

```
    let now = env.ledger().timestamp();

    if timestamp + threshold <= now {
        panic!("OracleError::OracleStalePrice");
    }

    price
}
```

Both `fetch_forward_price()` and `fetch_inverse_price()` depend on this function, which means all vault operations requiring price conversion will fail when either condition triggers.

Impact: Oracle unavailability blocks all price-dependent operations. This can occur if admin calls `disable_oracle()`, external oracle stops updating (triggering the staleness check), or the underlying `rwa_oracle` contract becomes unavailable.

The most significant impact is on **Liquidity Token (LT) holders**. LT positions have a fixed maturity window during which holders must withdraw. If the oracle becomes unavailable during this window, LT holders cannot exit their positions in time and may miss their redemption period entirely — potentially losing access to their funds or being forced to accept unfavorable terms after the window expires.

While admin can mitigate some scenarios (re-enable oracle, increase threshold), there is no user-accessible escape path during oracle downtime.

Recommended Mitigation: Consider implementing a fallback oracle as secondary price source

STBL: Acknowledged. Currently, the system uses a single oracle. However, the oracle is fully replaceable per asset, and the design allows for future migration to a multi-oracle setup to mitigate oracle dependency risks.

7.5 Informational

7.5.1 Unused local UPGRADER_ROLE in per asset contracts

Description: Constructors of AssetVault, AssetIssuer, and AssetYieldDistributor grant UPGRADER_ROLE to the caller via local AccessControl:

```
AccessControl::grant_role(&env, &caller, &caller, &String::from_str(&env, UPGRADER_ROLE));
```

However, upgrade() in these contracts checks the role against Registry's AccessControl, not the local one:

```
let reg = AccessControlClient::new(&env, &registry);
if !reg.has_role(&caller, &role) { ... }
```

The locally granted role is never read, resulting in wasted storage writes.

Affected locations:

- asset-vault
- asset-issuer
- asset-yield-distributor

Recommended Mitigation: Remove AccessControl::grant_role() for UPGRADER_ROLE from per-asset constructors.

STBL: Fixed in commit 7754c24e.

Cyfrin: Verified.

7.5.2 Unused Pause Functionality in Asset Contracts

Description: The AssetVault, AssetIssuer, and AssetYieldDistributor contracts initialize the pauser and expose pause()/unpause() functions via the impl_pauser_for! macro, but never check the pause state:

```
// asset-vault/src/lib.rs:114
Pauser::init_pauser(&env);

// asset-vault/src/lib.rs:182
impl_pauser_for!(AssetVault, PAUSE_ROLE);
```

```
// asset-issuer/src/lib.rs:86
Pauser::init_pauser(&env);

// asset-issuer/src/lib.rs:377
impl_pauser_for!(AssetIssuer, PAUSE_ROLE);
```

```
// asset-yield-distributor/src/lib.rs:156
Pauser::init_pauser(&env);

// asset-yield-distributor/src/lib.rs:430
impl_pauser_for!(AssetYieldDistributor, PAUSE_ROLE);
```

None of these contracts call Pauser::when_not_paused() in any function. The pause state can be toggled but has no effect on contract operations.

The Solidity implementation does not include pause functionality in Vault, Issuer, or YieldDistributor contracts — only in the token contracts.

Recommended Mitigation: Either remove the unused pause functionality from these contracts to match the Solidity implementation and remove dead code, or add Pauser::when_not_paused(&env) checks to critical functions if pause functionality is intentionally desired.

STBL: Fixed in commit bfcb1205.

Cyfrin: Verified.

7.5.3 Vault Solvency Check Blocks User Withdrawals After Yield Distribution

Description: The vault's `withdraw_erc20()` reverts when asset value falls below obligations:

```
// asset-vault/src/lib.rs:252-257
if usd_value_of_deposit < vault.deposit_value_usd {
    panic!("STBL_Asset_InsufficientVaultValue");
}
```

This check was added to fix a Solidity audit finding about underflow on negative rebasing. Cyfrin noted the tradeoff: *"the current fix reverts when vault becomes insolvent... this causes denial of service for later users attempting withdrawal."*

While prices are expected to only increase, negative rebasing can occur if the underlying issuer (e.g., ONDO) is forced to sell bonds at market price below purchase price.

Additionally, `distribute_yield()` extracts the entire price differential, leaving the vault at exact parity (`oracle_price * asset_deposit_net  deposit_value_usd`). Any subsequent price drop triggers the block.

LT tokens have a strict withdrawal window — after `yield_duration` but before `duration` expires. If blocked during this window, users cannot withdraw and their funds are forfeited to treasury via `withdrawExpired()`.

Unlike PT users who can wait for recovery, LT users face permanent loss due to their deadline.

Impact: Low — unlikely given price growth assumption, but LT users face permanent fund forfeiture if insolvency coincides with their withdrawal window.

Recommended Mitigation: Consider one of:

- Retain a small buffer during `distribute_yield()` instead of extracting the entire differential (most dangerous moment is immediately after yield distribution)
- Document this behavior clearly for LT depositors

STBL: Acknowledged. We are selecting our assets based on this assumption only. No negative / price downtrend will be there. So, yield will be non negative only.

7.5.4 Withdraw Fee Locked at Deposit Time

Description: When a user deposits, the current `withdraw_fee` percentage is read from the registry and stored in the YLD token metadata. This locked value is then used at withdrawal time, regardless of any fee changes that occurred in between:

```
// asset-issuer/src/lib.rs:419-420 - fee captured at deposit
meta.withdraw_fee_amount =
    (meta.stable_value_gross * meta.fees.withdraw_fee as u128) / FEES_CONSTANT as u128;

// asset-vault/src/lib.rs:260 - locked value used at withdrawal
let withdraw_asset_value = oracle_client.fetch_inverse_price(
    &(metadata.stable_value_net + metadata.haircut_amount - metadata.withdraw_fee_amount),
);
```

This means if admin changes the withdraw fee in the registry after a user deposits, the user still pays the old fee amount.

Impact: Fee changes do not apply retroactively to existing positions. If withdraw fees are lowered, existing users pay more than new depositors for the same service. If fees are raised, existing users pay less than intended under the new fee structure. This creates inconsistent treatment of users based on their deposit timing rather than current protocol parameters.

Recommended Mitigation: Document this behavior clearly so users understand their withdraw fee is locked at deposit time if this is intentional. Alternatively, if dynamic fees are desired, calculate the withdraw fee at withdrawal time using current registry values.

STBL: Fixed in commit e4c59782.

Cyfrin: Verified.

7.6 Gas Optimization

7.6.1 Redundant `is_active` Checks in Deposit and Withdraw Flows

Description: Both deposit and withdraw operations perform redundant `is_active()` checks across multiple contracts. Each check requires a cross-contract call to Registry, storage read, and deserialization of `AssetDefinition`.

During deposit:

1. `AssetIssuer::internal_deposit()` calls `assert_setup_done()` which checks `is_active()` (asset-issuer:127)
2. `AssetVault::deposit_erc20()` calls `assert_issuer()` which checks `is_active()` (asset-vault:159)
3. `StblCore::put()` checks `is_active()` (core:141)

During withdraw:

1. `AssetIssuer::internal_withdraw()` calls `assert_setup_done()` which checks `is_active()` (asset-issuer:127)
2. `AssetVault::withdraw_erc20()` calls `assert_issuer()` which checks `is_active()` (asset-vault:159)
3. `StblCore::exit()` checks `is_active()` (core:209)

Each operation performs 3 identical checks. The `AssetVault` and `StblCore` contracts already verify that the caller is an authorized issuer via `assert_issuer()` and `is_issuer()` respectively. Since only the issuer contract can call these functions, and the issuer already validated `is_active()` at entry, the downstream checks are redundant.

Recommended Mitigation: Remove `is_active()` checks from `AssetVault` and `StblCore`. The single check in `AssetIssuer::assert_setup_done()` is sufficient since downstream contracts already enforce that only the authorized issuer can call them.

STBL: Fixed in 32338a88.

Cyfrin: Verified.